

Fundamentos de Analítica

Taller 1

2026-2

1. Comandos DDL, DQL, DML, DCL y TCL en SQL. Para cada uno de estos tipos de comandos:
 - a) Defina brevemente en qué consiste cada grupo de comandos (DDL, DQL, DML, DCL, TCL).
 - b) Enumere los comandos principales que pertenezcan a cada grupo.
 - c) Explique para que sirven cada uno de estos comandos del numeral pasado.
 - d) Muestre un ejemplo de cada uno. Como se vería el código y cuál sería el resultado.
2. ¿Qué son las window functions?. Para responder, realiza lo siguiente:
 - a) Defina qué es una window function y cómo se diferencia de una función de agregación tradicional.
 - b) Explica la utilidad de PARTITION BY y ORDER BY dentro de las window functions. ¿Qué función cumple cada una y cómo influyen en el resultado?
 - c) Explique al menos cuatro ejemplos de window functions que usen explícitamente las cláusulas PARTITION BY y ORDER BY. Por ejemplo, ROW_NUMBER(), RANK(), SUM() OVER(), AVG() OVER(), etc.
 - d) Incluye para cada ejemplo:
 - a. El fragmento de código SQL con la window function y sus particiones/ordenamientos.
 - b. Una explicación de qué realiza la consulta y cómo se comportan las filas según el PARTITION BY y el ORDER BY.
 - c. Un breve ejemplo de tabla y cómo quedarían los resultados después de aplicar la función de ventana.
 - e) ¿Cómo identificaría usted que se requiere una window function para una consulta en particular?
3. En clase se explicó brevemente el concepto de JOIN, el cual permite combinar información de dos o más tablas relacionando sus columnas. En SQL también existen

Fundamentos de Analítica

Taller 1

2026-2

operadores que permiten combinar los resultados de diferentes consultas en el sentido de las filas.

¿Cuáles son los comandos u operadores SQL que permiten realizar este tipo de combinación? Explique brevemente cada uno y presente un ejemplo en el que se evidencie su uso, mostrando tanto las tablas o consultas involucradas como el resultado final obtenido.

4. Las siguientes preguntas/solicitudes simulan la manera como una persona del lado del negocio normalmente le pide información a un analista o a un ingeniero de datos:

Usuarios y registros recientes

Se necesita el listado de contactos de aquellos usuarios mayores de 30 años que se unieron este año, ordenados por fecha de registro, del más antiguo al más nuevo. Solo es necesario mostrar los primeros 5 registros encontrados. En este listado debe aparecer mínimamente el nombre, el correo, el número celular, la dirección y la fecha de registro.

Gasto acumulado por cliente

Entregue un resumen de cuánto ha gastado cada cliente en total, pero solo de quienes hayan superado los 700 dólares en su historial de compras.

Ciudades activas

Sería de interés para el negocio saber en qué ciudades potentes se tiene presencia: genere la lista de ubicaciones únicas donde viven usuarios que hayan realizado al menos dos pedidos.

Segmentación demográfica

Se requiere de un desglose del número de usuarios en función de su grupo de edad: jóvenes, adultos y seniors, según los rangos: menores de 25, entre 25 y 40, mayores de 40.

Informe combinado de usuarios y pedidos

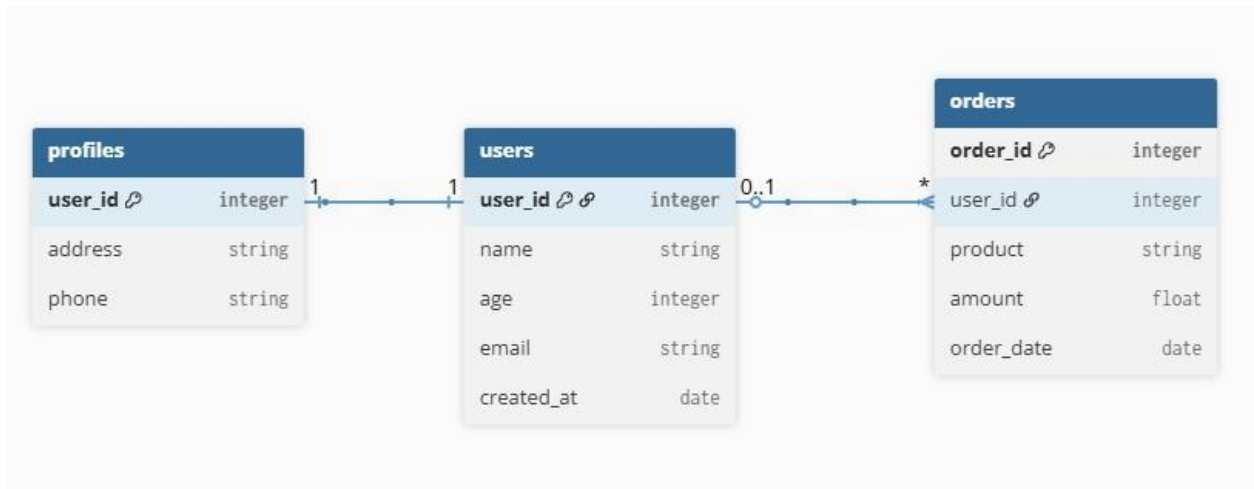
Genere un reporte que incluya los datos de contacto y toda la información personal de cada usuario, junto con sus pedidos más recientes. Muéstrelos ordenados cronológicamente.

Para poder cumplir con lo anterior se debe guiar por la base de datos db_minittest.sqlite3 y el siguiente diagrama:

Fundamentos de Analítica

Taller 1

2026-2



Utilice SQL ya sea desde Python o desde herramientas para SQL como DBeaver con el fin de responder estas preguntas de negocio.

5. Modelos Estrella y Copo de Nieve

- Describa las principales diferencias entre ambos modelos, teniendo en cuenta aspectos como la estructura de las tablas, el nivel de normalización, la redundancia de datos y la complejidad de las consultas.
- Mencione las principales ventajas y desventajas del modelo estrella frente al modelo copo de nieve, y viceversa.
- Explique en qué escenarios o situaciones es más conveniente utilizar un modelo estrella y en cuáles sería recomendable utilizar un modelo copo de nieve.
- Incluya un ejemplo sencillo, mediante un esquema o conjunto de tablas, que permita evidenciar cómo se representaría la misma información utilizando ambos modelos.

6. Dimensiones de Tiempo y Geografía en Modelos Analíticos

- Describa la importancia de la dimensión de tiempo en un modelo estrella o copo de nieve. ¿Qué tipos de atributos suelen incluirse en una dimensión de tiempo?
- Describa la importancia de la dimensión geográfica en un modelo estrella o copo de nieve. ¿Qué tipos de atributos suelen incluirse en una dimensión geográfica?
- Incluya un ejemplo sencillo del diseño de una tabla de dimensión de tiempo y una tabla de dimensión geográfica, así como la forma en que se relacionan con una tabla de hechos.

7. Usando la tabla sales.csv:

1. Normalización:

- Aplique el proceso de normalización desde la 1NF hasta la 4NF si lo considera necesario.
- Rediseñe la estructura de la base de datos y presente el código DDL completo del modelo resultante, incluyendo los tipos de datos y las relaciones necesarias.
- Utilice cualquier herramienta gráfica para mostrar el modelo final. Una recomendación es: <https://dbdiagram.io/>

2. Modelo estrella:

- A partir del modelo normalizado, diseñe un modelo estrella o copo de nieve apto para análisis de datos.
- Presente el código DDL (CREATE TABLE) completo para este modelo, especificando tipos de datos y relaciones entre tablas.
- Utilice cualquier herramienta gráfica para mostrar el modelo final.

Los siguientes puntos son considerados **EXTRA**, es decir, no son obligatorios, solo se realizan en caso tal de que se desee profundizar más sobre el tema:

Particiones e Índices en Bases de Datos Relacionales

- b) Explique qué es la partición de una tabla en una base de datos relacional. ¿Cuáles son las ventajas de utilizar particiones?
- c) Mencione y describa al menos dos tipos de particionamiento (por ejemplo, particionamiento por rango, por lista, por hash, etc.).
- d) Explique qué es un índice en una base de datos. ¿Para qué se utiliza y cómo mejora el desempeño de las consultas?
- e) Describa la diferencia entre índices únicos y no únicos.
- f) Incluya un ejemplo SQL de cómo crear una tabla particionada y un índice sobre ella.
- g) Explique brevemente cómo el uso de particiones e índices puede combinarse para optimizar consultas sobre grandes volúmenes de datos. ¿Cuándo se debe crear un índice o una partición?

Fundamentos de Analítica

Taller 1

2026-2

Vistas, Vistas Materializadas y Procedimientos Almacenados en Bases de

Datos Relacionales

- a) Explique qué es una vista (VIEW) en una base de datos relacional. ¿Cuál es su propósito y qué ventajas ofrece su utilización?
- b) Describa la diferencia entre una vista convencional y una vista materializada. ¿Cómo se almacena y consulta la información en cada una de ellas?
- c) Explique en qué situaciones puede ser conveniente utilizar una vista materializada en lugar de una vista convencional. ¿Qué ventajas y desventajas presenta en términos de almacenamiento, rendimiento y actualización de los datos?
- d) Explique qué es un procedimiento almacenado (Stored Procedure) y cuál es su utilidad dentro de una base de datos. Mencione algunas ventajas de utilizar procedimientos almacenados para implementar operaciones o lógica de negocio.
- e) Incluya un ejemplo SQL de creación y utilización de una vista, una vista materializada y un procedimiento almacenado. Para cada caso, muestre la consulta utilizada y un ejemplo de cómo se accede o ejecuta el objeto creado.
- f) Explique brevemente cómo las vistas, las vistas materializadas y los procedimientos almacenados pueden utilizarse conjuntamente para mejorar la organización, seguridad, reutilización de código y desempeño de una base de datos. ¿En qué situaciones sería recomendable utilizar cada uno de estos objetos?

Taller 1 realizado por Julián Alzate Aristizábal

Respuestas

1. Comandos DDL, DQL, DML, DCL y TCL en SQL

a) Definición de cada uno de los comandos:

- DDL (Data Definition language) son comandos que definen y modifican la estructura de los objetos que están en la base de datos, como tablas, esquemas y vistas.
- DQL (Data Query Language) son comandos que se dedican a consultar y recuperar información almacenada.
- DML (Data Manipulation Language) son comandos que manipulan el contenido de las tablas, como insertar, eliminar o actualizar filas.
- DCL (Data Control Language) son comandos que gestionan el acceso a los objetos de la base de datos, brindando permisos y control a los usuarios que deban ver esa información.
- TCL (Transaction Control Language) son comandos que definen y controlan las transacciones, es decir, administran cambios realizados por instrucciones DML (Data Manipulation Language) en una base de datos.

b) y c) Comandos principales de cada grupo y para qué sirven.

- **DDL**
 - **CREATE:** crea nuevos objetos como tablas, vistas, índices, esquemas, bases de datos.
 - **ALTER:** modifica la estructura de un objeto ya existente. Agregando o eliminando columnas, cambiando tipos de datos y añadiendo restricciones.
 - **DROP:** elimina por completo un objeto y su estructura de la base de datos.
- **DQL**
 - **SELECT:** recupera filas y columnas de una o varias tablas; se combina con otros comandos como WHERE, GROUP BY, HAVING, ORDER BY, JOIN, subconsultas, etc.
- **DML**
 - **INSERT:** agrega una o varias filas nuevas a una tabla.
 - **UPDATE:** modifica el valor de una o varias columnas en las filas que cumplen una condición.
 - **DELETE:** elimina filas de una tabla que cumplen con una condición dada.
- **DCL**
 - **GRANT:** otorga permisos específicos (SELECT, INSERT, UPDATE, etc.) a un usuario o rol sobre un objeto.
 - **REVOKE:** retira permisos previamente otorgados a un usuario o rol.

Fundamentos de Analítica

Taller 1

2026-2

- **TCL**

- **COMMIT:** Guarda de manera permanente los cambios realizados en la transacción actual.

d) Ejemplos de código y resultado. Para los códigos se le pidió a la IA (CLAUDE) que creara una base de datos pequeña para poder correr los comandos de SQL. La base de dato se llama taller_sql.sqlite3.

- **DDL**

- **CREATE**

```
query = """
CREATE TABLE IF NOT EXISTS bonificaciones (
    bonificacion_id INTEGER PRIMARY KEY,
    empleado_id INTEGER,
    monto REAL,
    fecha TEXT
)
"""

cur.execute(query)
conn.commit()

query2 = """
SELECT * FROM bonificaciones
"""

pd.read_sql_query(query2, conn)
```

✓ 0.0s

bonificacion_id	empleado_id	monto	fecha
-----------------	-------------	-------	-------

- **ALTER**

```
query = """
ALTER TABLE bonificaciones ADD COLUMN activo INTEGER DEFAULT 1
"""

cur.execute(query)
conn.commit()
```

✓ 0.0s

bonificacion_id	empleado_id	monto	fecha	activo
-----------------	-------------	-------	-------	--------

- **DROP**

Fundamentos de Analítica

Taller 1

2026-2

```
query = """
DROP TABLE IF EXISTS bonificaciones

"""

cur.execute(query)
conn.commit()

query2 = """
SELECT * FROM bonificaciones
"""

pd.read_sql_query(query2, conn)
```

[28] 0.0s

```
...
OperationalError                                Traceback (most recent call last)
File ~\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra...
2744 try:
-> 2745     cur.execute(sql, *args)
      2746     return cur

OperationalError: no such table: bonificaciones
```

- DQL
 - SELECT

```
query = """
SELECT nombre, departamento, ciudad FROM empleados
"""

pd.read_sql_query(query, conn)
```

	nombre	departamento	ciudad
0	Ana Ramírez	Ventas	Bogotá
1	Carlos Vega	Marketing	Medellín
2	Diana Soto	Ventas	Cali
3	Felipe Cruz	IT	Bogotá
4	Gabriela Ríos	IT	NaN
5	Hugo Mendoza	Ventas	Medellín
6	Isabel Nava	Marketing	Bogotá
7	Jorge Paredes	Finanzas	Cali
8	Karla Duarte	Finanzas	Bogotá
9	Luis Ortega	IT	Medellín

- DML
 - INSERT

```
query = """
INSERT INTO bonificaciones (empleado_id, monto, fecha, activo)
VALUES (1, 150000, '2024-06-15', 1)
"""

cur.execute(query)
conn.commit()

pd.read_sql_query("SELECT * FROM bonificaciones", conn)
```

✓ 0.0s

	bonificacion_id	empleado_id	monto	fecha	activo
0		1	150000.0	2024-06-15	1

Fundamentos de Analítica

Taller 1

2026-2

○ UPDATE

```
query = """
UPDATE bonificaciones
SET monto = 200000
WHERE empleado_id = 1
"""

cur.execute(query)
conn.commit()

pd.read_sql_query("SELECT * FROM bonificaciones WHERE empleado_id = 1", conn)
```

✓ 0.0s

bonificacion_id	empleado_id	monto	fecha	activo	
0	1	1	200000.0	2024-06-15	1

○ DELETE

```
query = """
DELETE FROM bonificaciones
WHERE empleado_id = 1
"""

cur.execute(query)
conn.commit()

pd.read_sql_query("SELECT * FROM bonificaciones", conn)
```

✓ 0.0s

bonificacion_id	empleado_id	monto	fecha	activo
-----------------	-------------	-------	-------	--------

- DCL: Las siguientes imágenes fueron obtenidas de imágenes de Google.

○ GRANT

```
GRANT CONTROL ON SCHEMA::Person TO Demologin;
EXECUTE AS USER = 'Demologin';
SELECT * FROM fn_my_permissions ('Person', 'SCHEMA');
REVERT;
GO
```

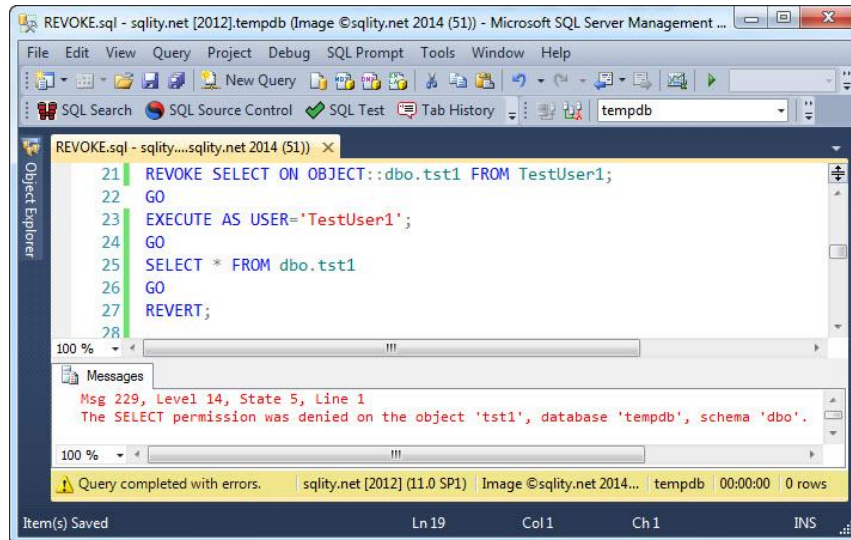
	entity_name	subentity_name	permission_name
1	Person		SELECT
2	Person		INSERT
3	Person		UPDATE
4	Person		DELETE
5	Person		REFERENCES
6	Person		EXECUTE
7	Person		CREATE SEQUENCE
8	Person		VIEW CHANGE TRACKING
9	Person		VIEW DEFINITION
10	Person		ALTER
11	Person		TAKE OWNERSHIP
12	Person		CONTROL

Fundamentos de Analítica

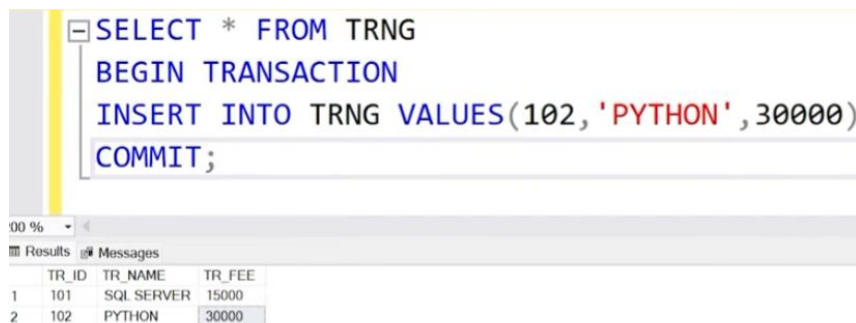
Taller 1

2026-2

○ REVOKE



- TCL
 - COMMIT



2. ¿Qué son las window functions?

- a. Una **Window Function** es una función que realiza un cálculo sobre un conjunto de filas relacionadas con la fila actual, denominada ventana.

A diferencia de una función de agregación tradicional (SUM, AVG, COUNT usadas con GROUP BY), no colapsa esas filas en una sola. Cada fila del resultado conserva su propia identidad y se le añade el valor calculado sobre su ventana. Se puede entender de la siguiente manera:

Con la window function (como OVER) puedo mantener mis N filas intactas a la vez que agrego una columna con una función (sea suma, resta, etc). Mientras que con una función de agregación tradicional como GROUP BY, pasé de tener mis N filas a N-1 o N-i filas por grupo, perdiendo el detalle de las demás.

b. Window PARTITION BY y ORDER BY

PARTITION BY: el comando divide el conjunto de filas en grupos, llamados particiones, según los valores de una o más columnas. La window function se calcula de forma independiente dentro de cada partición, reiniciándose para cada nuevo grupo. Es el equivalente, dentro de una window function, a lo que GROUP BY hace para una agregación tradicional, con la diferencia de que aquí no se colapsan las filas.

ORDER BY (dentro del OVER()): define el orden en el que se procesan las filas dentro de cada partición. Este orden es indispensable para funciones que dependen de una secuencia, como ROW_NUMBER(), RANK() o LAG()/LEAD(), y también determina el comportamiento por defecto del "marco" (frame) en funciones acumulativas como SUM() OVER(), que por defecto acumula desde el inicio de la partición hasta la fila actual.

Si no se especifica PARTITION BY, la ventana completa (todas las filas del resultado) se trata como una única partición. Si no se especifica ORDER BY, funciones como SUM() OVER() calculan el total de toda la partición en cada fila, en lugar de un acumulado progresivo

c. y d. Ejemplos de window functions

▪ ROW_NUMBER()

El comando asigna un número consecutivo (como 1, 2, 3, etc.) a cada fila, reiniciando el conteo en cada vendedor (PARTITION BY vendedor). Además, ordena de mayor a menor monto dentro de cada vendedor (ORDER BY monto DESC). En el siguiente ejemplo es útil para identificar, la venta más realizada por cada vendedor.

```
#ROW_NUMBER
query = """
SELECT
    v.venta_id,
    e.nombre AS vendedor,
    v.producto,
    v.cantidad * v.precio_unit AS monto,
    ROW_NUMBER() OVER (
        PARTITION BY e.nombre
        ORDER BY v.cantidad * v.precio_unit DESC
    ) AS num_venta
FROM ventas v
INNER JOIN empleados e ON e.empleado_id = v.empleado_id
"""

pd.read_sql_query(query, conn)

✓ 0.0s
```

	venta_id	vendedor	producto	monto	num_venta
0	1	Ana Ramírez	Laptop	3500000.0	1
1	11	Ana Ramírez	Monitor	1300000.0	2
2	2	Ana Ramírez	Mouse	135000.0	3
3	18	Carlos Vega	Silla Oficina	750000.0	1
4	6	Carlos Vega	Camiseta Corp	250000.0	2
5	3	Diana Soto	Silla Oficina	500000.0	1
6	7	Diana Soto	Teclado	180000.0	2
7	16	Diana Soto	Mouse	180000.0	3
8	4	Felipe Cruz	Monitor	650000.0	1
9	15	Felipe Cruz	Cuaderno	45000.0	2
10	17	Gabriela Ríos	Teclado	90000.0	1
11	5	Hugo Mendoza	Escritorio	480000.0	1
12	12	Hugo Mendoza	Camiseta Corp	125000.0	2
13	20	Isabel Nava	Laptop	3500000.0	1
14	9	Isabel Nava	Cuaderno	60000.0	2
15	14	Jorge Paredes	Escritorio	960000.0	1
16	19	Karla Duarte	Monitor	650000.0	1
17	10	Karla Duarte	Silla Oficina	250000.0	2
18	13	Luis Ortega	Laptop	3500000.0	1

▪ RANK()

El comando ordena de mayor a menor monto las filas de una columna y les asigna un puesto dentro de cada categoría (PARTITION BY categoría). Si dos filas tienen el mismo monto reciben el mismo ranking, y el siguiente puesto salta el número correspondiente. Se le hace al ranking a las categorías de ventas según su monto.

```
#RANK
query = """
SELECT
    v.venta_id,
    v.categoria,
    v.cantidad * v.precio_unit AS monto,
    RANK() OVER (
        PARTITION BY v.categoria
        ORDER BY v.cantidad * v.precio_unit DESC
    ) AS ranking
FROM ventas v
"""

pd.read_sql_query(query, conn)
```

✓ 0.0s

Fundamentos de Analítica

Taller 1

2026-2

	venta_id	categoria	monto	ranking
0	6	Merch	250000.0	1
1	12	Merch	125000.0	2
2	14	Mobiliario	960000.0	1
3	18	Mobiliario	750000.0	2
4	3	Mobiliario	500000.0	3
5	5	Mobiliario	480000.0	4
6	10	Mobiliario	250000.0	5
7	9	Papelería	60000.0	1
8	15	Papelería	45000.0	2
9	1	Tecnología	3500000.0	1
10	13	Tecnología	3500000.0	1
11	20	Tecnología	3500000.0	1
12	11	Tecnología	1300000.0	4
13	8	Tecnología	700000.0	5
14	4	Tecnología	650000.0	6
15	19	Tecnología	650000.0	6
16	7	Tecnología	180000.0	8
17	16	Tecnología	180000.0	8
18	2	Tecnología	135000.0	10
19	17	Tecnología	90000.0	11

▪ SUM() OVER()

El comando calcula un acumulado del monto para cada vendedor. Gracias al ORDER BY venta_id dentro del OVER(), el marco por defecto va "desde el inicio de la partición hasta la fila actual", por lo que cada fila muestra la suma de sus ventas hasta ese punto (sin incluir las siguientes).

```
#SUMA
query = """
SELECT
    v.venta_id,
    e.nombre AS vendedor,
    v.fecha_venta,
    v.cantidad * v.precio_unit AS monto,
    SUM(v.cantidad * v.precio_unit) OVER (
        PARTITION BY e.nombre
        ORDER BY v.fecha_venta
    ) AS acumulado
FROM ventas v
INNER JOIN empleados e ON e.empleado_id = v.empleado_id
"""

pd.read_sql_query(query, conn)
✓ 0.0s
```

Fundamentos de Analítica

Taller 1

2026-2

	venta_id	vendedor	fecha_venta	monto	acumulado
0	1	Ana Ramírez	2024-01-15	3500000.0	3635000.0
1	2	Ana Ramírez	2024-01-15	135000.0	3635000.0
2	11	Ana Ramírez	2024-05-15	1300000.0	4935000.0
3	6	Carlos Vega	2024-01-30	250000.0	250000.0
4	18	Carlos Vega	2024-08-12	750000.0	1000000.0
5	3	Diana Soto	2024-02-20	500000.0	500000.0
6	7	Diana Soto	2024-04-02	180000.0	680000.0
7	16	Diana Soto	2024-07-15	180000.0	860000.0
8	4	Felipe Cruz	2023-11-05	650000.0	650000.0
9	15	Felipe Cruz	2024-07-10	45000.0	695000.0
10	17	Gabriela Ríos	2024-08-05	90000.0	90000.0
11	5	Hugo Mendoza	2024-03-10	480000.0	480000.0
12	12	Hugo Mendoza	2024-06-01	125000.0	605000.0
13	9	Isabel Nava	2024-03-18	60000.0	60000.0
14	20	Isabel Nava	2024-09-01	3500000.0	3560000.0
15	14	Jorge Paredes	2024-02-28	960000.0	960000.0
16	19	Karla Duarte	2024-01-22	650000.0	650000.0
17	10	Karla Duarte	2024-05-01	250000.0	900000.0
18	13	Luis Ortega	2024-06-20	3500000.0	3500000.0

▪ AVG() OVER()

El comando o más bien función, calcula el promedio de TODA la partición (todas las ventas de esa categoría) y lo repite en cada fila. Esto permite comparar cada venta individual contra el promedio de su categoría sin perder el detalle por fila, algo que un simple GROUP BY no permitiría hacer en la misma consulta.

```
#AVG - OVER
query = """
SELECT
    v.venta_id,
    v.categoria,
    v.cantidad * v.precio_unit AS monto,
    ROUND(AVG(v.cantidad * v.precio_unit) OVER (PARTITION BY v.categoria), 2) AS promedio_categoria
FROM ventas v
"""

pd.read_sql_query(query, conn)
✓ 0.1s
```

Fundamentos de Analítica

Taller 1

2026-2

venta_id	categoria	monto	promedio_categoria
0	6 Merch	250000.0	187500.00
1	12 Merch	125000.0	187500.00
2	3 Mobiliario	500000.0	588000.00
3	5 Mobiliario	480000.0	588000.00
4	10 Mobiliario	250000.0	588000.00
5	14 Mobiliario	960000.0	588000.00
6	18 Mobiliario	750000.0	588000.00
7	9 Papelería	60000.0	52500.00
8	15 Papelería	45000.0	52500.00
9	1 Tecnología	3500000.0	1307727.27
10	2 Tecnología	135000.0	1307727.27
11	4 Tecnología	650000.0	1307727.27
12	7 Tecnología	180000.0	1307727.27
13	8 Tecnología	700000.0	1307727.27
14	11 Tecnología	1300000.0	1307727.27
15	13 Tecnología	3500000.0	1307727.27
16	16 Tecnología	180000.0	1307727.27
17	17 Tecnología	90000.0	1307727.27
18	19 Tecnología	650000.0	1307727.27
19	20 Tecnología	3500000.0	1307727.27

d. ¿Cuándo se requiere una Window function?

Es conveniente usar una window function cuando se necesita agregar un cálculo sin perder el detalle de cada fila original, cuando quiero comparar una fila con otra relacionada a una medición en particular, o si necesito rankear o posicionar resultados de varios grupos para hallar las mejores 3 ventas o los ganadores de un concurso.

En pocas palabras si lo que necesito o me preguntan combina un “por” o un “cada” con la necesidad de conservar todas las filas de la tabla, en ese caso siempre necesitaré una window function como PARTITION BY/ORDER BY para conservar todas las filas originales. Pero si en lugar solo quiero resumir el resultado con ciertas condiciones usaría GROUP BY. Ejemplo: Si necesito calcular el promedio de la nota de una materia de todos los estudiantes y mostrar el nombre de cada uno, usaría una window function. Pero si solo necesito el promedio de un solo estudiante o solo los estudiantes llamados JUAN, no usaría una window function (GROUP BY).

3. Operadores de conjuntos SQL

Aquellas operaciones en SQL que permiten combinar resultados de dos o más consultas en sentido vertical (por filas) se les llama operadores de conjuntos. A diferencia de los JOINT, que se utilizan principalmente para relacionar información de diferentes tablas mediante una condición común, combinando la información principalmente en sentido horizontal, es decir, agregando columnas de las tablas relacionadas al resultado.

Los principales Operadores de conjuntos son (con ejemplo):

- **UNION:** Combina las filas de dos consultas y elimina automáticamente las filas duplicadas resultantes.

```
# UNION, empleados que vendieron en el 1er semestre 0 en el 2do, sin repetir
query = """
SELECT empleado_id FROM ventas WHERE fecha_venta < '2024-07-01'
UNION
SELECT empleado_id FROM ventas WHERE fecha_venta >= '2024-07-01'
"""
pd.read_sql_query(query, conn)
✓ 0.0s
```

	empleado_id
0	NaN
1	1.0
2	2.0
3	3.0
4	4.0
5	5.0
6	6.0
7	7.0
8	8.0
9	9.0
10	10.0

- **UNION ALL:** Combina las filas de dos consultas SIN eliminar duplicados; es más rápida que UNION porque no necesita comparar ni duplicar filas.

Fundamentos de Analítica

Taller 1

2026-2

```
# UNION ALL, lo mismo que pasa con UNION pero con duplicados (el que vendió en ambos semestres aparece 2 veces)
query = """
SELECT empleado_id FROM ventas WHERE fecha_venta < '2024-07-01'
UNION ALL
SELECT empleado_id FROM ventas WHERE fecha_venta >= '2024-07-01'
"""

pd.read_sql_query(query, conn)
```

✓ 0.0s

	empleado_id
0	1.0
1	1.0
2	3.0
3	4.0
4	6.0
5	2.0
6	3.0
7	NaN
8	7.0
9	9.0
10	1.0
11	6.0
12	10.0
13	8.0
14	9.0
15	4.0
16	3.0
17	5.0
18	2.0
19	7.0

- **INTERSECT:** Devuelve únicamente las filas que aparecen en AMBAS consultas (la intersección de los dos conjuntos de resultados).

```
#INTERSECT, muestra los empleados que vendieron en AMBOS semestres
query = """
SELECT empleado_id FROM ventas WHERE fecha_venta < '2024-07-01'
INTERSECT
SELECT empleado_id FROM ventas WHERE fecha_venta >= '2024-07-01'
"""

pd.read_sql_query(query, conn)
```

✓ 0.0s

	empleado_id
0	2
1	3
2	4
3	7

- **EXCEPT:** Devuelve las filas que están en la primera consulta, pero NO en la segunda (la diferencia entre los dos conjuntos).

```
#EXCEPT, muestra los empleados que vendieron en el 1er semestre pero NO en el 2do
query = """
SELECT empleado_id FROM ventas WHERE fecha_venta < '2024-07-01'
EXCEPT
SELECT empleado_id FROM ventas WHERE fecha_venta >= '2024-07-01'
"""
pd.read_sql_query(query, conn)
```

✓ 0.0s

	empleado_id
0	NaN
1	1.0
2	6.0
3	8.0
4	9.0
5	10.0

4. Preguntas/solicitudes de negocio

Usuarios y registros recientes

Si se usaba como referencia la fecha actual (2026) la consulta devolvía datos vacíos, por lo que se toma como referencia el año 2025.

```
import sqlite3
import pandas as pd

conn2 = sqlite3.connect('db_minitest.sqlite3')
cur = conn2.cursor()
```

✓ 0.5s

```
# Usuarios y registros recientes. Nota: si se usaba como referen
query = """
SELECT
    u.name,
    u.email,
    p.phone,
    p.address,
    u.created_at
FROM users u
INNER JOIN profiles p ON p.user_id = u.user_id
WHERE u.age > 30
    AND strftime('%Y', u.created_at) = '2025'
ORDER BY u.created_at ASC
LIMIT 5
"""
pd.read_sql_query(query, conn2)
```

✓ 0.0s

	name	email	phone	address	created_at
0	Diana	diana@example.com	555-3456	321 Pine St	2025-03-15
1	Edward	edward@example.com	555-7890	654 Cedar Ave	2025-04-10
2	Hannah	hannah@example.com	555-0123	135 Willow Dr	2025-06-01

Gasto acumulado por cliente

```
# Gasto acumulado por cliente
query = """
SELECT
    u.user_id,
    u.name,
    SUM(o.amount) AS gasto_total
FROM users u
INNER JOIN orders o ON o.user_id = u.user_id
GROUP BY u.user_id, u.name
HAVING SUM(o.amount) > 700
ORDER BY gasto_total DESC
"""

pd.read_sql_query(query, conn2)
```

✓ 0.0s

	user_id	name	gasto_total
0	1	Alice	1225.5

Ciudades activas

```
# Ciudades activas
query = """
SELECT p.address
FROM profiles p
INNER JOIN orders o ON o.user_id = p.user_id
GROUP BY p.user_id, p.address
HAVING COUNT(o.order_id) >= 2
"""

pd.read_sql_query(query, conn2)
```

✓ 0.0s

	address
0	123 Main St
1	321 Pine St
2	654 Cedar Ave
3	987 Birch Rd
4	135 Willow Dr

Segmentación demográfica

```
# Segmentación demográfica
query = """
SELECT
    CASE
        WHEN age < 25 THEN 'Joven'
        WHEN age BETWEEN 25 AND 40 THEN 'Adulto'
        ELSE 'Senior'
    END AS grupo_edad,
    COUNT(*) AS num_usuarios
FROM users
GROUP BY grupo_edad
"""

pd.read_sql_query(query, conn2)
```

✓ 0.0s

	grupo_edad	num_usuarios
0	Adulto	3
1	Joven	2
2	Senior	3

Informe combinado de usuarios y pedidos

```
#Informe combinado de usuarios y pedidos
query = """
SELECT
    u.user_id,
    u.name,
    u.email,
    u.age,
    p.address,
    p.phone,
    o.order_id,
    o.product,
    o.amount,
    o.order_date
FROM users u
INNER JOIN profiles p ON p.user_id = u.user_id
INNER JOIN orders o ON o.user_id = u.user_id
ORDER BY o.order_date DESC
"""

pd.read_sql_query(query, conn2)
```

✓ 0.0s

	user_id	name	email	age	address	phone	order_id	product	amount	order_date
0	8	Hannah	hannah@example.com	42	135 Willow Dr	555-0123	1015	Charger	20.0	2025-06-06
1	8	Hannah	hannah@example.com	42	135 Willow Dr	555-0123	1014	Smartwatch	250.0	2025-06-05
2	6	Fiona	fiona@example.com	27	987 Birch Rd	555-2345	1012	Filter	30.0	2025-05-17
3	6	Fiona	fiona@example.com	27	987 Birch Rd	555-2345	1011	Bag	50.0	2025-05-16
4	6	Fiona	fiona@example.com	27	987 Birch Rd	555-2345	1010	Lens	200.0	2025-05-15
5	5	Edward	edward@example.com	55	654 Cedar Ave	555-7890	1009	Tripod	80.0	2025-04-16
6	5	Edward	edward@example.com	55	654 Cedar Ave	555-7890	1008	Camera	500.0	2025-04-15
7	4	Diana	diana@example.com	35	321 Pine St	555-3456	1007	Webcam	55.0	2025-04-01
8	4	Diana	diana@example.com	35	321 Pine St	555-3456	1006	Headphones	85.0	2025-03-25
9	4	Diana	diana@example.com	35	321 Pine St	555-3456	1005	Tablet	450.0	2025-03-20
10	2	Bob	bob@example.com	22	456 Elm St	555-5678	1003	Keyboard	75.0	2025-03-01
11	1	Alice	alice@example.com	30	123 Main St	555-1234	1002	Mouse	25.5	2025-02-15
12	1	Alice	alice@example.com	30	123 Main St	555-1234	1001	Laptop	1200.0	2025-02-01
13	3	Charlie	charlie@example.com	45	789 Oak St	555-9012	1004	Monitor	300.0	2024-12-20
14	7	George	george@example.com	19	246 Spruce Ln	555-6789	1013	Memory Card	45.0	2024-11-05

5. Modelos Estrella y Copo de Nieve

a. Diferencias principales

MODELO ESTRELLA	MODELO COPO DE NIEVE
• Es más rápido para lectura y consulta de datos. • Consultas más simples • Tabla de hechos centrada rodeada de dimensiones planas, es decir, una tabla por dimensión. • Dimensiones desnormalizadas (baja normalización, 1NF/2NF) • Mayor redundancia dentro de cada dimensión	• Es más lento en la lectura, pero ahorra espacio de almacenamiento. • Consultas más complejas. • Tabla de hechos central rodeada de dimensiones que a su vez se descompone en subtablas relacionadas. • Dimensiones normalizadas (más cercanas a 3NF) • Menor redundancia; los atributos jerárquicos viven en tablas separadas

b. Ventajas y desventajas

Modelo Estrella

- **Ventajas:** Consultas más simples y rápidas de escribir y ejecutar, mejor rendimiento en herramientas de BI. Es más sencillo de entender.
- **Desventajas:** Mayor redundancia de datos debido a la desnormalización, lo que puede requerir más espacio de almacenamiento.

Modelo Copo de Nieve:

- **Ventajas:** Menor redundancia, menos espacio de almacenamiento, y mejor integridad de datos debido a la normalización, jerarquías de atributos más simples.
- **Desventajas:** consultas más complejas, se traduce a más JOINS. Múltiples JOINS puede afectar el rendimiento de las consultas. Es más difícil de entender.

c. Escenarios de Usos recomendados

Es recomendado usar el **modelo Estrella** cuando se necesita velocidad en las consultas y simplicidad para crear reportes o dashboards de BI, como en análisis de ventas, inventarios, clientes, etc.

Fundamentos de Analítica

Taller 1

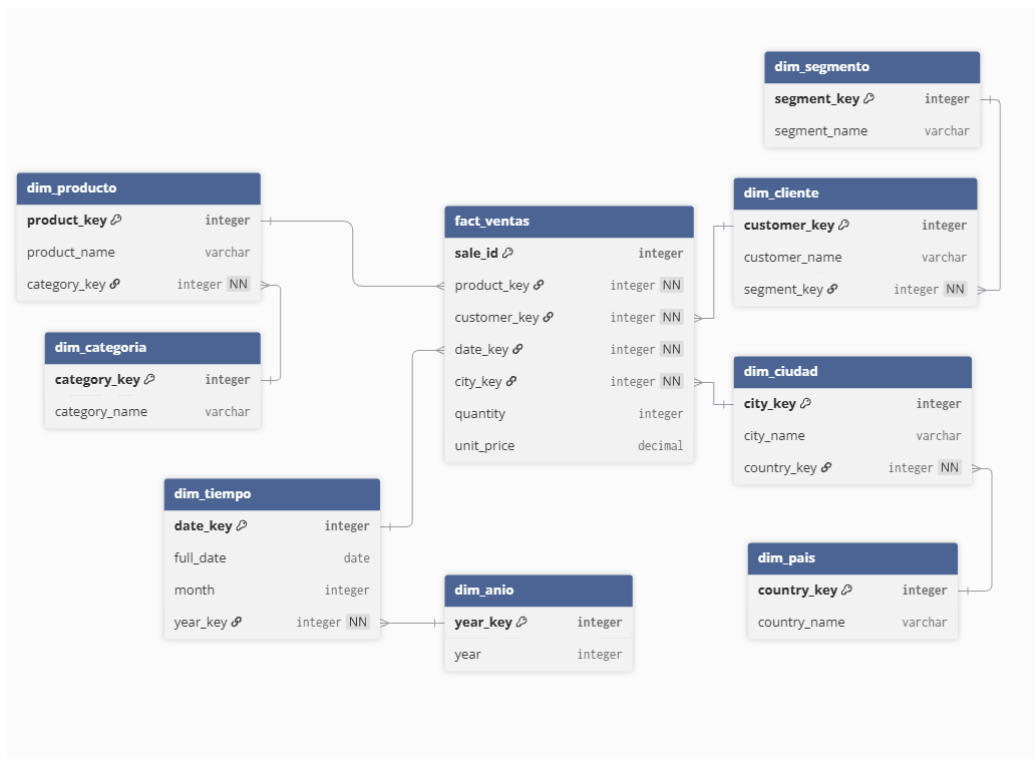
2026-2

Por otro lado, es recomendado usar el **modelo Copo de Nieve** cuando se desea reducir la redundancia y optimizar el almacenamiento. Además, es recomendable utilizarlo cuando las dimensiones tienen jerarquías complejas y muy detalladas, como en consultas de ubicaciones: país, departamento, ciudad, zona, barrio, etc.

d. Imágenes:

Supongamos que tenemos una base de datos de facturación de ventas. En el modelo estrella, dim_producto guarda la categoría directamente como un atributo repetido en cada fila; en el modelo copo de nieve, la categoría se separa en su propia tabla dim_categoria y dim_producto solo guarda la llave foránea.

- **Copo de Nieve**

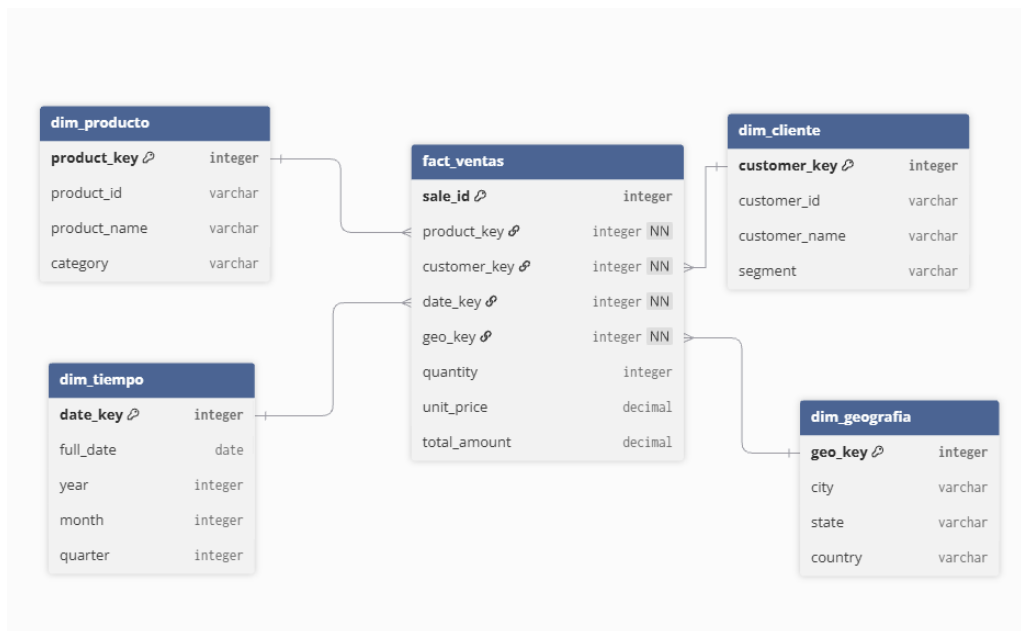


- **Estrella**

Fundamentos de Analítica

Taller 1

2026-2



6. Dimensiones de Tiempo y Geografía en Modelos Analíticos

- La dimensión de tiempo es importante en modelos analíticos (modelos estrella/copo de nieve) porque permite organizar, comparar y analizar los datos según diferentes periodos de tiempo. Permitiendo responder preguntas como ¿cuánto se vendió en el mes? ¿Qué día hubo más ventas?

Los atributos que suelen incluirse en una dimensión de tiempo son, por ejemplo:

- Fecha completa (FULL DATE)
 - Día
 - Día de la semana
 - Semana
 - Mes
 - Año
 - Trimestre
 - Data key con la fecha completa
- La dimensión geográfica también es importante en modelos analíticos, ya que permite organizar, comparar y analizar los datos según diferentes ubicaciones geográficas. Lo que ayuda a responder preguntas como ¿Qué país tiene más población? ¿Qué territorio tiene más ingresos?

Fundamentos de Analítica

Taller 1

2026-2

Los atributos que suelen incluirse en una dimensión geográfica son, por ejemplo:

- País
- Estado o departamento
- Ciudad o municipio
- Barrio
- Coordenadas (Latitud, longitud, altitud)
- Dirección
- Código Postal

- c. Un ejemplo de una tabla de dimensión de tiempo y geográfica relacionadas con una tabla de hechos puede ser el siguiente, tomando los atributos mencionados en los puntos anteriores:



Creación de las tablas en SQL, están vacías ya que no se pedían con filas adentro:

Fundamentos de Analítica

Taller 1

2026-2

```
conn = sqlite3.connect(':memory:')
cur = conn.cursor()
cur.executescript("""
CREATE TABLE dim_tiempo (
    date_key      INTEGER PRIMARY KEY,  -- formato yyyymmdd
    full_date     DATE NOT NULL,
    dia           INTEGER,
    dia_semana    VARCHAR(10),
    semana        INTEGER,
    mes           INTEGER,
    nombre_mes    VARCHAR(15),
    trimestre     INTEGER,
    año           INTEGER,
    es_fin_de_semana BOOLEAN,
    es_festivo     BOOLEAN
);

CREATE TABLE dim_geografia (
    geo_key      INTEGER PRIMARY KEY,
    ciudad       VARCHAR(50),
    departamento VARCHAR(50),
    pais         VARCHAR(50),
    region       VARCHAR(50),
    codigo_postal VARCHAR(10),
    latitud      DECIMAL(9,6),
    longitud     DECIMAL(9,6)
);

CREATE TABLE fact_ventas (
    sale_id      INTEGER PRIMARY KEY,
    date_key     INTEGER REFERENCES dim_tiempo(date_key),
    geo_key      INTEGER REFERENCES dim_geografia(geo_key),
    product_key  INTEGER,
    customer_key INTEGER,
    quantity     INTEGER,
    unit_price   DECIMAL(12,2),
    total_amount DECIMAL(12,2)
);
""")
conn.commit()
pd.read_sql_query("SELECT name FROM sqlite_master WHERE type='table'", conn)
```

Vista de tablas individualmente:

	name
0	dim_tiempo
1	dim_geografia
2	fact_ventas


```
pd.read_sql_query("SELECT * FROM dim_tiempo",conn)
✓ 0.0s
```

date_key	full_date	dia	dia_semana	semana	mes	nombre_mes	trimestre	año	es_fin_de_semana	es_festivo
----------	-----------	-----	------------	--------	-----	------------	-----------	-----	------------------	------------


```
pd.read_sql_query("SELECT * FROM dim_geografia",conn)
✓ 0.0s
```

geo_key	ciudad	departamento	pais	region	codigo_postal	latitud	longitud
---------	--------	--------------	------	--------	---------------	---------	----------


```
pd.read_sql_query("SELECT * FROM fact_ventas",conn)
✓ 0.0s
```

sale_id	date_key	geo_key	product_key	customer_key	quantity	unit_price	total_amount
---------	----------	---------	-------------	--------------	----------	------------	--------------

7. Normalización y Modelos usando sales.csv

a. Normalización

La primera forma normal 1NF ya se aplica/cumple al importar el csv, todos los atributos son atómicos y cada fila esta identificada por una clave general que es `sale_id`

La segunda forma 2NF también ya cumple desde la importación del csv, en este caso `sale_id` es una clave primaria. Esta identifica por sí sola cada venta, y no hay necesidad de combinar más columnas para identificar la fila, por lo que no hay que eliminar dependencias de alguna clave compuesta.

Tabla original sales.csv:

	sale_id	sale_date	customer_id	customer_name	customer_city	product_id	product_name	category	quantity	unit_price	rep_id	rep_name	region
0	1001	1/07/2025	C001	María Gómez	Bogotá	P100	Teclado Mecánico	Periféricos	2	1200	R10	Juan Pérez	Andina
1	1002	2/07/2025	C002	Andrés Rodríguez	Medellín	P101	Mouse Óptico	Periféricos	1	455	R11	Laura Sánchez	Pacífico
2	1003	2/07/2025	C001	María Gómez	Bogotá	P200	Monitor 24"	Pantallas	1	3500	R10	Juan Pérez	Andina
3	1004	3/07/2025	C003	Luis Martínez	Cali	P300	SSD 1TB	Almacenamiento	3	900	R12	Camila Torres	Pacífico
4	1005	4/07/2025	C004	Ana Beltrán	Barranquilla	P100	Teclado Mecánico	Periféricos	1	1200	R11	Laura Sánchez	Pacífico
5	1006	5/07/2025	C005	Pedro Castillo	Bogotá	P300	SSD 1TB	Almacenamiento	2	900	R10	Juan Pérez	Andina
6	1007	6/07/2025	C006	Carla Mendoza	Cartagena	P101	Mouse Óptico	Periféricos	4	455	R12	Camila Torres	Pacífico
7	1008	7/07/2025	C007	Diego López	Bucaramanga	P200	Monitor 24"	Pantallas	2	3500	R11	Laura Sánchez	Pacífico
8	1009	8/07/2025	C008	Sofía García	Bogotá	P400	Webcam HD	Periféricos	1	700	R10	Juan Pérez	Andina
9	1010	9/07/2025	C009	Javier Ruiz	Medellín	P500	Disco Duro 2TB	Almacenamiento	2	1100	R11	Laura Sánchez	Pacífico
10	1011	10/07/2025	C010	Natalia Jiménez	Cali	P200	Monitor 24"	Pantallas	1	3500	R12	Camila Torres	Pacífico
11	1012	11/07/2025	C002	Andrés Rodríguez	Medellín	P100	Teclado Mecánico	Periféricos	3	1200	R11	Laura Sánchez	Pacífico
12	1013	12/07/2025	C003	Luis Martínez	Cali	P101	Mouse Óptico	Periféricos	5	455	R12	Camila Torres	Pacífico
13	1014	13/07/2025	C004	Ana Beltrán	Barranquilla	P200	Monitor 24"	Pantallas	1	3500	R10	Laura Sánchez	Pacífico
14	1015	14/07/2025	C005	Pedro Castillo	Bogotá	P400	Webcam HD	Periféricos	2	700	R11	Juan Pérez	Andina
15	1016	15/07/2025	C006	Carla Mendoza	Cartagena	P500	Disco Duro 2TB	Almacenamiento	1	1100	R12	Camila Torres	Pacífico
16	1017	16/07/2025	C007	Diego López	Bucaramanga	P300	SSD 1TB	Almacenamiento	3	900	R11	Laura Sánchez	Pacífico
17	1018	17/07/2025	C008	Sofía García	Bogotá	P100	Teclado Mecánico	Periféricos	1	1200	R10	Juan Pérez	Andina
18	1019	18/07/2025	C009	Javier Ruiz	Medellín	P101	Mouse Óptico	Periféricos	2	455	R11	Laura Sánchez	Pacífico
19	1020	19/07/2025	C010	Natalia Jiménez	Cali	P200	Monitor 24"	Pantallas	1	3500	R12	Camila Torres	Pacífico

La tercera forma 3NF se debe de aplicar para sales.csv. Hay varias columnas que no dependen de `sale_id`, como `customer_name` y `customer_city` que dependen de `customer_id`. Pero que `customer_id` si depende de `sale_id`, lo que genera redundancias. Por lo tanto, se procede a separar las columnas en 5 tablas, cada una con su llave primaria:

- `customers(customer_id, customer_name, customer_city)`, `customer_id` es la clave primaria.
- `products(product_id, product_name, category)`, `product_id` es la clave primaria.
- `reps(rep_id, rep_name, region)`, `rep_id` es la clave primaria.
- `sales(sale_id, sale_date, customer_id, rep_id)` `sale_id` es la clave primaria, y `customer_id` y `rep_id` son las claves foraneas.

Fundamentos de Analítica

Taller 1

2026-2

- sale_items(sale_id, product_id, quantity, unit_price) en este caso sale_id y product_id serian una clave primaria compuesta ya que no producto_id no es suficiente para diferenciar una fila de otra.

```
cur.executescript("""
CREATE TABLE customers (
  customer_id  VARCHAR(10) PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  customer_city VARCHAR(60)
);

CREATE TABLE products (
  product_id   VARCHAR(10) PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
  category     VARCHAR(50)
);

CREATE TABLE reps (
  rep_id   VARCHAR(10) PRIMARY KEY,
  rep_name VARCHAR(100) NOT NULL,
  region   VARCHAR(50)
);

CREATE TABLE sale_items (
  sale_id      INTEGER NOT NULL REFERENCES sales(sale_id),
  product_id   VARCHAR(10) NOT NULL REFERENCES products(product_id),
  quantity     INTEGER NOT NULL CHECK (quantity > 0),
  unit_price   DECIMAL(12,2) NOT NULL CHECK (unit_price >= 0),
  PRIMARY KEY (sale_id, product_id)
);

CREATE TABLE sales (
  sale_id      INTEGER PRIMARY KEY,
  sale_date    DATE NOT NULL,
  customer_id  VARCHAR(10) NOT NULL REFERENCES customers(customer_id),
  rep_id       VARCHAR(10) NOT NULL REFERENCES reps(rep_id)
);

""")
```

Fundamentos de Analítica

Taller 1

2026-2

```

"""
customers = df[['customer_id', 'customer_name', 'customer_city']].drop_duplicates().sort_values('customer_id')
products = df[['product_id', 'product_name', 'category']].drop_duplicates().sort_values('product_id')
reps = df[['rep_id', 'rep_name', 'region']].drop_duplicates().sort_values('rep_id')
sales = df[['sale_id', 'sale_date', 'customer_id', 'rep_id']].drop_duplicates().sort_values('sale_id')
sale_items = df[['sale_id', 'product_id', 'quantity', 'unit_price']].sort_values('sale_id')

customers.to_sql('customers', conn3, if_exists='append', index=False)
products.to_sql('products', conn3, if_exists='append', index=False)
reps.to_sql('reps', conn3, if_exists='append', index=False)
sales.to_sql('sales', conn3, if_exists='append', index=False)
sale_items.to_sql('sale_items', conn3, if_exists='append', index=False)

conn3.commit()

```

```
pd.read_sql_query("SELECT * FROM customers", conn3)
```

	customer_id	customer_name	customer_city
0	C001	María Gómez	Bogotá
1	C002	Andrés Rodríguez	Medellín
2	C003	Luis Martínez	Cali
3	C004	Ana Beltrán	Barranquilla
4	C005	Pedro Castillo	Bogotá
5	C006	Carla Mendoza	Cartagena
6	C007	Diego López	Bucaramanga
7	C008	Sofía García	Bogotá
8	C009	Javier Ruiz	Medellín
9	C010	Natalia Jiménez	Cali

```
pd.read_sql_query("SELECT * FROM products", conn3)
```

	product_id	product_name	category
0	P100	Teclado Mecánico	Periféricos
1	P101	Mouse Óptico	Periféricos
2	P200	Monitor 24"	Pantallas
3	P300	SSD 1TB	Almacenamiento
4	P400	Webcam HD	Periféricos
5	P500	Disco Duro 2TB	Almacenamiento

```
pd.read_sql_query("SELECT * FROM reps", conn3)
```

	rep_id	rep_name	region
0	R10	Juan Pérez	Andina
1	R11	Laura Sánchez	Pacífico
2	R12	Camila Torres	Pacífico

Fundamentos de Analítica

Taller 1

2026-2

```
pd.read_sql_query("SELECT * FROM sale_items", conn3)
```

	sale_id	product_id	quantity	unit_price
0	1001	P100	2	1200
1	1002	P101	1	455
2	1003	P200	1	3500
3	1004	P300	3	900
4	1005	P100	1	1200
5	1006	P300	2	900
6	1007	P101	4	455
7	1008	P200	2	3500
8	1009	P400	1	700
9	1010	P500	2	1100
10	1011	P200	1	3500
11	1012	P100	3	1200
12	1013	P101	5	455
13	1014	P200	1	3500
14	1015	P400	2	700
15	1016	P500	1	1100
16	1017	P300	3	900
17	1018	P100	1	1200
18	1019	P101	2	455
19	1020	P200	1	3500

```
pd.read_sql_query("SELECT * FROM sales", conn3)
```

	sale_id	sale_date	customer_id	rep_id
0	1001	1/07/2025	C001	R10
1	1002	2/07/2025	C002	R11
2	1003	2/07/2025	C001	R10
3	1004	3/07/2025	C003	R12
4	1005	4/07/2025	C004	R11
5	1006	5/07/2025	C005	R10
6	1007	6/07/2025	C006	R12
7	1008	7/07/2025	C007	R11
8	1009	8/07/2025	C008	R10
9	1010	9/07/2025	C009	R11
10	1011	10/07/2025	C010	R12
11	1012	11/07/2025	C002	R11
12	1013	12/07/2025	C003	R12
13	1014	13/07/2025	C004	R11
14	1015	14/07/2025	C005	R10
15	1016	15/07/2025	C006	R12
16	1017	16/07/2025	C007	R11
17	1018	17/07/2025	C008	R10
18	1019	18/07/2025	C009	R11
19	1020	19/07/2025	C010	R12

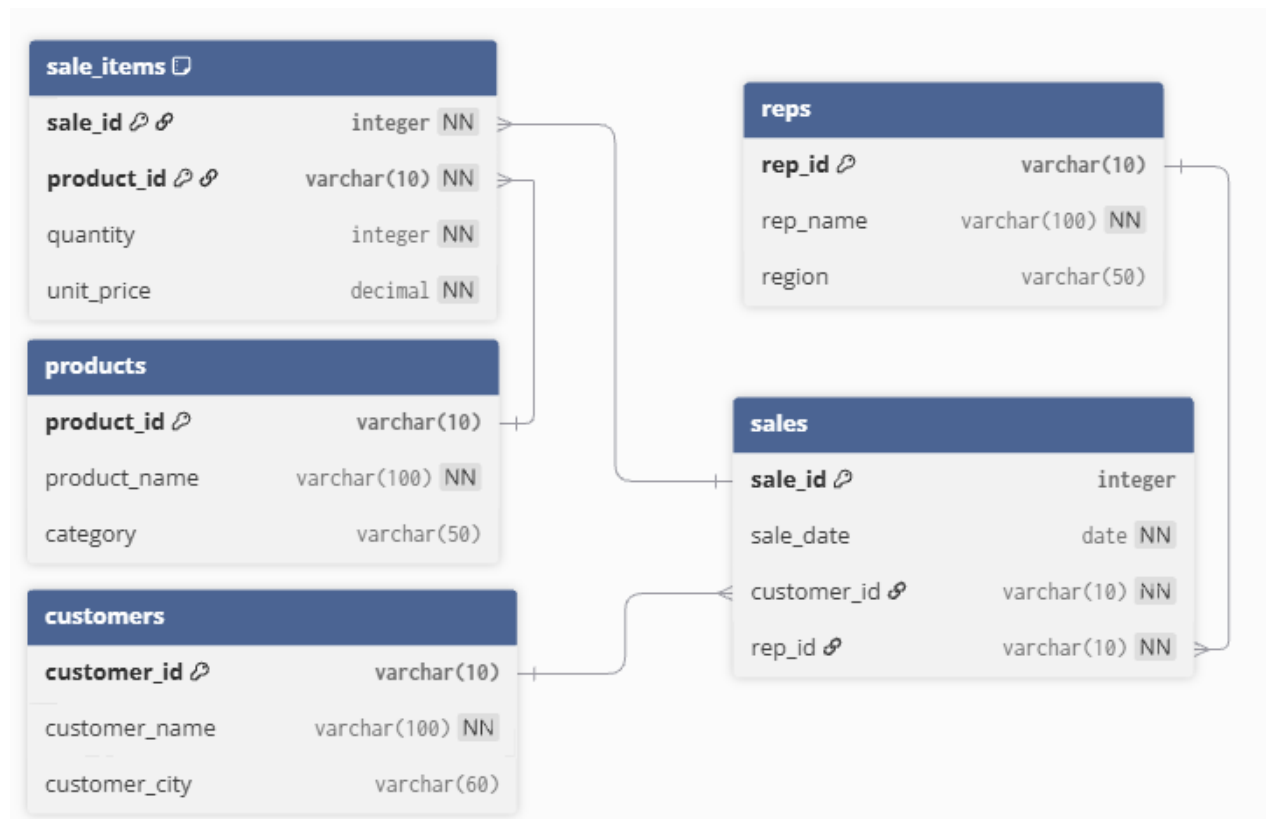
Fundamentos de Analítica

Taller 1

2026-2

Para la cuarta forma normalizada 4NF no tendríamos que usar esta descomposición adicional, ya que no se presentan más atributos que dependan de las llaves primarias y que a su vez sea independiente. Cada cliente tiene una sola ciudad, cada producto una sola categoría, cada representante una sola región, y cada línea de sale_items ya asocia un único producto con su cantidad y precio

Gráfica de Modelo Normalizado



b. Modelo estrella

Para este modelo vamos a cambiar el número de tablas normalizadas en el numeral anterior, y vamos también a nombrarlas como dimensiones y la tabla central, que sería sales, en la tabla de hechos. Esta tabla de hechos esta rodeada por 4 dimensiones dim_customer, dim_product, dim_rep y una tabla nueva de fechas que se llama dim_date.

Para fact_sales se añadió una columna nueva para tener un mejor análisis de la compra. Esta columna es el total_amount que sería el costo total de la compra que se saca multiplicando la cantidad por el precio unitario del producto.

- dim_customers(customer_id, customer_name, customer_city), customer_id es la clave primaria.
- dim_products(product_id, product_name, category), product_id es la clave primaria.
- dim_reps(rep_id, rep_name, region), rep_id es la clave primaria.
- dim_date(date_key, full_date) date_key es la clave primaria.
- fact_sales(sale_id, product_id, date_key, customer_id, rep_id, quantity, unit_price, total_amount). En este caso, sale_id y product_id forman una clave primaria compuesta, ya que ninguna de las dos por sí sola es suficiente para identificar una fila de forma única. A parte, date_key, customer_id, rep_id y product_id son llaves foráneas hacia dim_date, dim_customers, dim_reps y dim_products respectivamente.

```
cur.executescript("""
CREATE TABLE dim_customer (
    customer_id VARCHAR(10) PRIMARY KEY,
    customer_name VARCHAR(100),
    customer_city VARCHAR(60)
);

CREATE TABLE dim_product (
    product_id VARCHAR(10) PRIMARY KEY,
    product_name VARCHAR(100),
    category VARCHAR(50)
);

CREATE TABLE dim_reps (
    rep_id VARCHAR(10) PRIMARY KEY,
    rep_name VARCHAR(100),
    region VARCHAR(50)
);

CREATE TABLE dim_date (
    date_key INTEGER PRIMARY KEY,
    full_date DATE
);

CREATE TABLE fact_sales (
    sale_id INTEGER NOT NULL,
    product_id VARCHAR(10) NOT NULL REFERENCES dim_product(product_id),
    date_key INTEGER REFERENCES dim_date(date_key),
    customer_id VARCHAR(10) REFERENCES dim_customer(customer_id),
    rep_id VARCHAR(10) REFERENCES dim_rep(rep_id),
    quantity INTEGER,
    unit_price DECIMAL(12,2),
    total_amount DECIMAL(12,2),
    PRIMARY KEY (sale_id, product_id)
);

""")
conn3.commit()
```

Fundamentos de Analítica

Taller 1

2026-2

Vamos a llenar la tabla `dim_date` con las fechas de la tabla `sales` de la columna `sale_date`.

```
cur.execute("""
INSERT INTO dim_date (date_key, full_date)
SELECT DISTINCT
    CAST(strftime('%Y%m%d', sale_date) AS INTEGER),
    sale_date
FROM sales;
""")
conn3.commit()

pd.read_sql_query("SELECT * FROM dim_date ORDER BY date_key", conn3)
```

✓ 0.0s

	date_key	full_date
0	1	1/07/2025
1	2	2/07/2025
2	3	3/07/2025
3	4	4/07/2025
4	5	5/07/2025
5	6	6/07/2025
6	7	7/07/2025
7	8	8/07/2025
8	9	9/07/2025
9	10	10/07/2025
10	11	11/07/2025
11	12	12/07/2025
12	13	13/07/2025
13	14	14/07/2025
14	15	15/07/2025
15	16	16/07/2025
16	17	17/07/2025
17	18	18/07/2025
18	19	19/07/2025

Ahora con el `INNER JOIN` vamos a enlazar la información de las filas de las tablas de dimensiones y de la tabla `sales` en nuestra tabla de hechos `fact_sales`.

Fundamentos de Analítica

Taller 1

2026-2

```
cur.execute("""
INSERT INTO fact_sales (sale_id, product_id, date_key, customer_id, rep_id, quantity, unit_price, total_amount)
SELECT
    s.sale_id,
    si.product_id,
    d.date_key,
    s.customer_id,
    s.rep_id,
    si.quantity,
    si.unit_price,
    si.quantity * si.unit_price
FROM sales s
INNER JOIN
    sale_items si ON si.sale_id = s.sale_id
INNER JOIN
    dim_date d ON d.full_date = s.sale_date
""")
conn3.commit()

pd.read_sql_query("SELECT * FROM fact_sales", conn3)
```

✓ 0.0s

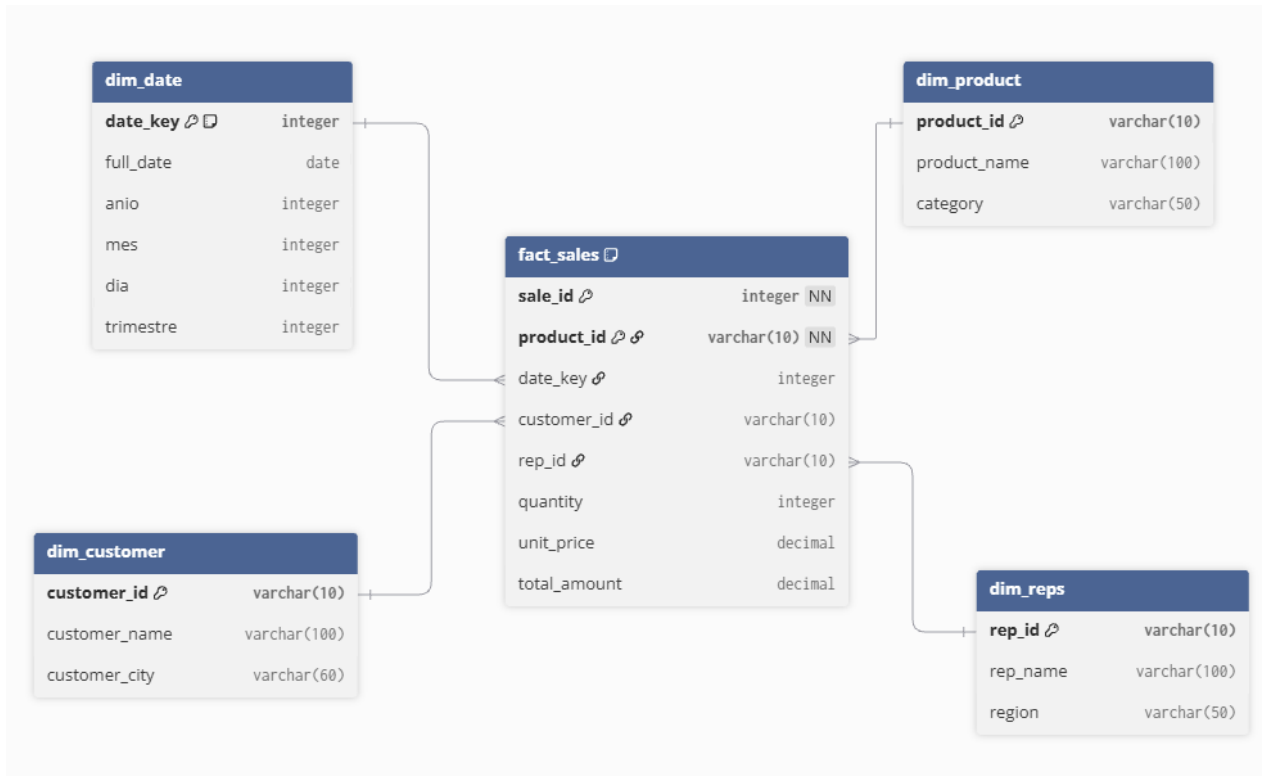
	sale_id	product_id	date_key	customer_id	rep_id	quantity	unit_price	total_amount
0	1001	P100	1	C001	R10	2	1200	2400
1	1002	P101	2	C002	R11	1	455	455
2	1003	P200	2	C001	R10	1	3500	3500
3	1004	P300	3	C003	R12	3	900	2700
4	1005	P100	4	C004	R11	1	1200	1200
5	1006	P300	5	C005	R10	2	900	1800
6	1007	P101	6	C006	R12	4	455	1820
7	1008	P200	7	C007	R11	2	3500	7000
8	1009	P400	8	C008	R10	1	700	700
9	1010	P500	9	C009	R11	2	1100	2200
10	1011	P200	10	C010	R12	1	3500	3500
11	1012	P100	11	C002	R11	3	1200	3600
12	1013	P101	12	C003	R12	5	455	2275
13	1014	P200	13	C004	R11	1	3500	3500
14	1015	P400	14	C005	R10	2	700	1400
15	1016	P500	15	C006	R12	1	1100	1100
16	1017	P300	16	C007	R11	3	900	2700
17	1018	P100	17	C008	R10	1	1200	1200
18	1019	P101	18	C009	R11	2	455	910
19	1020	P200	19	C010	R12	1	3500	3500

Fundamentos de Analítica

Taller 1

2026-2

Gráfica de Modelo Estrella



ARCHIVOS ADJUNTOS:

1. Respuestas del taller
2. Sales.csv
3. db_minitest.sqlite3
4. taller_sql.sqlite3
5. Página de códigos en Python: Taller_SQL_notas.ipynb